DEVFLOW — PROJECT CONTEXT MASTER DOCUMENT

Upload this document to an AI coding assistant at the beginning of a new session. It is intended to give the AI enough context to understand what is being built, why it is being built, the planned architecture, current phase, and project rules.

1. Project Identity

Project	DevFlow
Product type	Personal Operating System + Project / Client Management SaaS for developers and freelancers
Goal	Build a genuinely useful product that can be personally used while also serving as a strong real-world portfolio project.
Target build period	Approximately 1 month / 30 days
Stack	React, Tailwind CSS, Supabase, Git, GitHub
Current phase	Week 1 — Foundation

2. Product Vision

DevFlow is a command center for a developer/freelancer. The user creates an account, manages projects and clients, breaks projects into tasks and milestones, plans work for each day, tracks progress, manages contract/payment information, and receives configurable reminders. The product should be simple enough for daily use but engineered like a real SaaS application.

3. Core Modules

Module	Purpose	Phase
Authentication	Signup, login, logout, session persistence, protected routes.	Week 1
Profile & Settings	Profile information, timezone, reminder enabled/disabled, custom reminder time.	Week 1
Dashboard	High-level view of active work and useful summaries.	Foundation Week 1; real data later
Projects	Project CRUD, status, progress, dates, budget, notes.	Week 2
Tasks	Tasks, priority, status, due dates, project relationship, completion.	Week 2
My Day / Daily Plan	Show and organize work planned for today; support multi-day execution.	Week 2
Milestones	Group project work into meaningful milestones.	Week 2
Clients	Client/company/contact details and associated projects.	Week 3
Payments	Contract amount, payments made, remaining balance, payment history.	Week 3
Notifications	Daily reminders, overdue tasks, deadlines and payment-related alerts.	Week 4

4. Important Product Requirements

- The daily reminder time must be user-configurable. It must never be hard-coded to a fixed time such as 6 PM.
- Reminder settings should include enabled/disabled and a preferred time such as 6:00 PM, 7:30 PM, or 9:00 PM.
- Projects contain tasks and can span multiple days.
- The daily workflow should eventually let the user see exactly what needs to be done today.
- Client financial tracking should support contract amount, amount paid, remaining balance and payment history.
- The app should be multi-user ready and user data must be isolated securely.
- Supabase Row Level Security (RLS) should be treated as a core authorization layer when real application data is added.

- The product should be dogfooded: once features exist, DevFlow should be used to plan and track the development of DevFlow itself.

5. Week 1 — Foundation Only

Week 1 is intentionally narrow. Do not prematurely build the full project/task/client/payment/notification system. The objective is a clean, stable foundation that later features can build on.

Day	Work	Definition of Done
1	React + Vite, Tailwind, Git/GitHub, folder structure, responsive shell.	Running app, repository and basic layout.
2	UI foundation: sidebar, navbar, dashboard static cards.	Professional app shell and responsive UI.
3	Supabase setup + profiles table.	Database connection and initial user profile model.
4	Authentication: signup/login; optional password recovery.	User authentication works.
5	Protected routes, auth state, logout, session persistence.	Secure authenticated app flow.
6	Profile + Settings; save custom reminder preference.	User can edit profile and reminder setting.
7	Polish, responsive QA, loading/error states, console cleanup, final commit.	Stable Week 1 foundation.

6. Initial Data Model Direction

Week 1 profile model should contain approximately:

- id — profile/user identifier linked to Supabase Auth
- full_name — display name
- avatar_url — optional profile image
- timezone — user timezone for future scheduling
- reminder_enabled — boolean
- reminder_time — preferred daily reminder time
- created_at — creation timestamp

Future domain tables are expected to include projects, tasks, milestones, clients, payments, notifications and related history/activity. Do not create unnecessary complexity during Week 1.

7. Architecture Direction

Layer	Responsibility
React UI	Pages, components, forms, navigation, loading/error/empty states.
Feature/application logic	Business rules, validation, feature hooks/services and reusable state logic.
Supabase Auth	Identity, signup/login and sessions.
Supabase Database	Persistent application data.
RLS	Database-level authorization and user-data isolation.
Storage	Future avatars/files where needed.
Scheduled/background logic	Future reminder processing; exact mechanism decided when notifications are implemented.

8. Development Principles

- Real-world first: features must solve realistic developer/freelancer problems.
- Build usable vertical slices rather than many empty screens.
- Security by design: authentication and authorization are part of the architecture.
- Use existing products such as Linear, ClickUp or Asana only for inspiration; do not blindly copy them.
- Keep UI clean, responsive and understandable.
- Prefer maintainable architecture over clever or unnecessarily abstract code.
- Use Git meaningfully with commits that represent actual milestones.
- Do not add a feature just because it is technically interesting; protect the one-month scope.

9. AI Assistant Rules

When this document is given to an AI coding assistant, treat it as the project's baseline context. Before suggesting code, identify the current phase. Do not jump into later features unless explicitly requested.

- The user already knows React; avoid beginner React tutorials unless asked.
- Focus explanations on real-world engineering, architecture, database design, auth/security and product decisions.
- Assume React + Tailwind + Supabase + Git/GitHub unless the user explicitly changes the stack.
- When suggesting a feature, label whether it belongs to the current phase or a later phase.
- If a requested change conflicts with the current scope, explain the conflict instead of silently expanding the project.
- Prefer production-minded but appropriately simple solutions for a one-month portfolio SaaS.

10. Week 1 Definition of Done

- Project runs locally without errors.
- Git repository is initialized and organized.
- Responsive application shell exists.
- Supabase is connected safely.
- Profiles table exists with initial fields.
- Signup/login/logout/session persistence work.
- Protected routes work.
- Profile/settings can save custom reminder preference.
- Loading/error states are handled where appropriate.
- Avoidable console errors/warnings are cleaned up.

- Final Week 1 commit is made.

11. 30-Day Roadmap

Week 1: Foundation. Week 2: Projects, Tasks, My Day and Milestones. Week 3: Clients, Payments and richer Dashboard/Project Details. Week 4: Notifications, testing, UX polish, deployment and portfolio documentation.